



UNIVERSIDADE ESTADUAL PAULISTA
“JÚLIO DE MESQUITA FILHO”
Campus de São José do Rio Preto

REDES DE COMPUTADORES

Redes de Computadores (2375SCC) - 2025

Prof. Dr. Adriano Mauro Cansian

adriano.cansian@unesp.br

[linkedin.com/in/adrianocansian/](https://www.linkedin.com/in/adrianocansian/)

Curso de Redes de Computadores (I)

Adriano Mauro Cansian

Capítulo 2 Camada de Aplicação (Parte 1)

Licença de uso deste material (1)

unesp - SJRP



Estes slides foram produzidos e são disponibilizados sob a licença ["Creative Commons Atribuição Não Comercial SemDerivações 4.0 Internacional \(CC BY-NC-ND 4.0\)"](#).

Como dar o crédito apropriado:

Para qualquer reutilização ou distribuição, você deve deixar claro a terceiros os termos da licença a que se encontra submetida esta obra, e seu autor. A melhor maneira de fazer isso é citando a fonte e contato. Por exemplo:

Fonte: Adriano Mauro Cansian. **"Redes de Computadores – Camada de Aplicação (Parte 1)"**, São Paulo, SP. (2025).

Contato: adriano.cansian@unesp.br. Licenciamento Creative Commons [\(CC BY-NC-ND 4.0\)](#)

[linkedin.com/in/adrianocansian/](https://www.linkedin.com/in/adrianocansian/)

Fair Use: Este material pode usar citações e figuras de outros autores. Direitos reservados aos devidos autores. Todas as citações feitas em caráter didático.

Avisos: Você não tem de cumprir com os termos da licença relativamente a elementos do material que estejam no domínio público ou cuja utilização seja permitida por uma exceção ou limitação que seja aplicável. Não são dadas quaisquer garantias. A licença pode não lhe dar todas as autorizações necessárias para o uso pretendido. Por exemplo, outros direitos, tais como direitos de imagem, de privacidade ou direitos morais, podem limitar o uso do material.

Licença de uso deste material (2)

unesp - SJRP



- **Você tem o direito de:**

Compartilhar - copiar e redistribuir o material em qualquer suporte ou formato.

- **De acordo com os termos seguintes:**

- **(BY) Atribuição** - Você deve dar o crédito apropriado, prover um *link* para esta licença e indicar se mudanças foram feitas. Você deve fazê-lo em qualquer circunstância razoável, mas de nenhuma maneira que sugira que o licenciante apoia você ou o seu uso.
- **(NC) Não Comercial** - Você não pode usar o material para fins comerciais.
- **(ND) Sem Derivações** - Se você remixar, transformar ou criar a partir do material, você não pode distribuir o material modificado.
- **Sem restrições adicionais** - Você não pode aplicar termos jurídicos ou medidas de caráter tecnológico que restrinjam legalmente outros de fazerem algo que a licença permita.

Referência para estudo:

REDES DE COMPUTADORES E A INTERNET - 6ª Edição

James F. Kurose e Keith W. Ross. Edit. Pearson 2014.

ISBN: 9788581436777.

Capítulo 2 – Camada de Aplicação (6a. Ed.)



Capítulo 2: Camada de Aplicação

Metas do capítulo:

☐ Conhecer:

- Aspectos conceituais dos protocolos de **aplicação**.
- **Modelo cliente-servidor**.
- **Modelos de serviço**.

☐ Como:

- Estudar os **protocolos mais comuns** da camada da aplicação.

Veremos:

☐ Protocolos específicos:

- *HTTP*
- *FTP*
- Email - *SMTP*
- *DNS*
- *P2P - Torrents*.

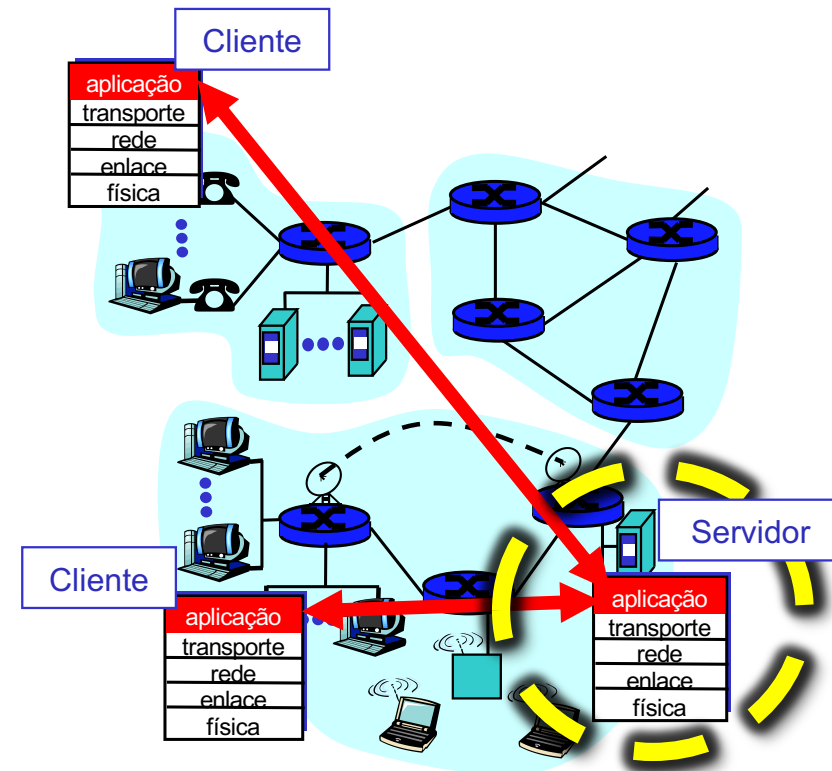
Conceitos Fundamentais

Aplicações:

Clientes, **servidores**, processos, **portas**, **sockets**, e mais...

Aplicações e protocolos da camada de aplicação

- ❑ Aplicações: são **processos distribuídos que se comunicam**.
 - **Trocam mensagens.**
 - **Rodam nos sistemas finais.**
 - Muitos exemplos:
 - E-mail, messengers, www, ssh, games, áudio & vídeo streams, IoT, VoIP, P2P, etc...



Protocolos de aplicação

❑ **Protocolo** da camada de aplicação:

- **Não é a aplicação.**

- É APENAS uma **parte** da aplicação.

O protocolo de aplicação define:

❑ **Mensagens trocadas.**

❑ **Ações** em resposta.

Os processos, em dois sistemas de borda diferentes, se comunicam **logicamente** trocando **mensagens**.

- ❑ Processo **emissor** **cria e emite** mensagens.

- ❑ Processo **receptor** **recebe essas** mensagens.

- E, eventualmente, **responde** emitindo mensagens de volta.

Protocolos definem:

□ Mensagens da camada de aplicação definem:

- **Tipos de mensagens.**

- Exemplo: de **requisição** e de **resposta**.

- **Sintaxe** dos tipos de mensagem.

- Define campos da mensagem, e como são **delimitados**.

- **Semântica dos campos.**

- Define o **significado da informação** dos campos.

- **Regras e Timing.**

- **Quando** e **como** um processo **envia** e **responde** mensagens.



SYNTAX



SEMANTICS



TIMING

(Todas as camadas definem tipos, sintaxe, semântica e regras)

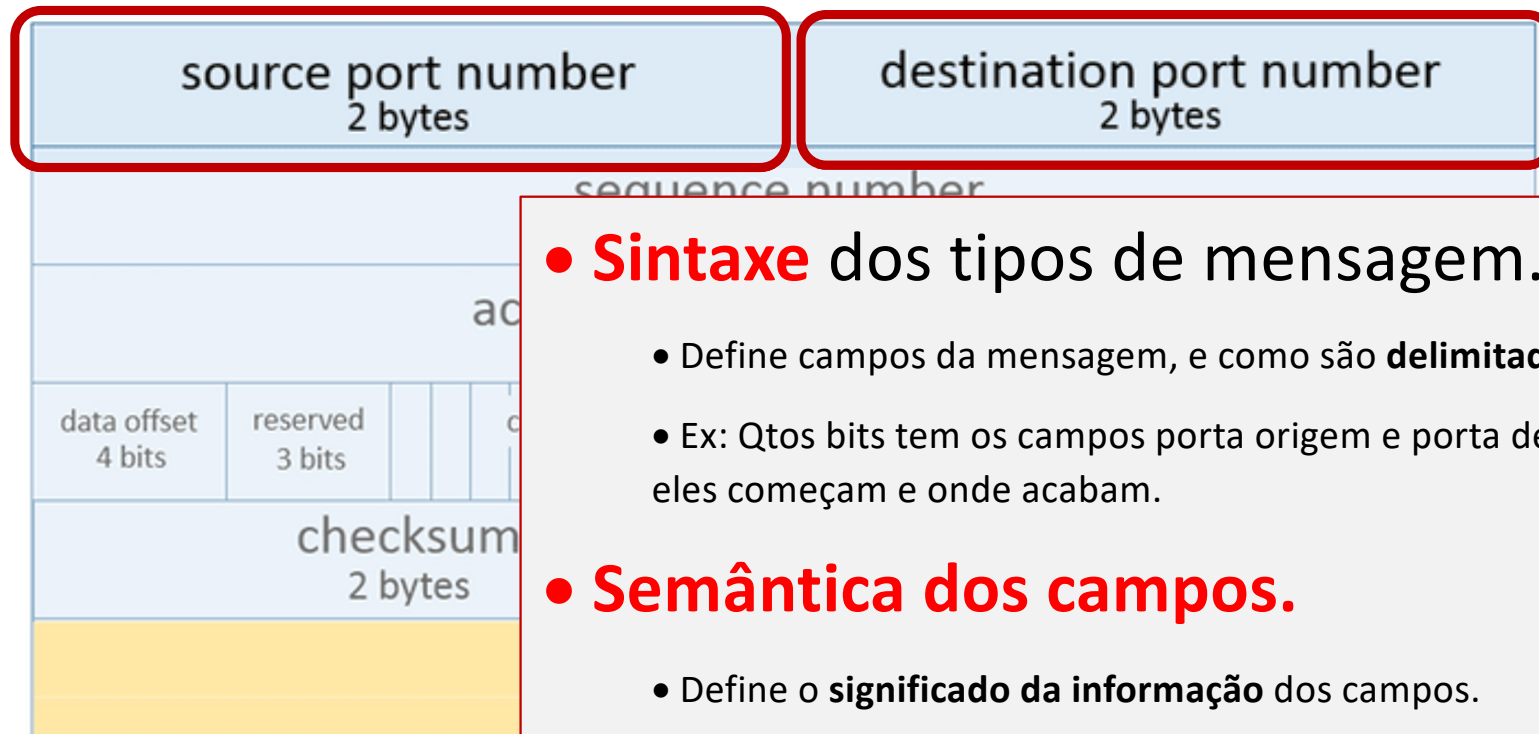
Exemplo:

Transmission Control Protocol (TCP) Header 20-60 bytes

source port number 2 bytes				destination port number 2 bytes			
sequence number 4 bytes							
acknowledgement number 4 bytes							
data offset 4 bits		reserved 3 bits		control flags 9 bits		window size 2 bytes	
checksum 2 bytes				urgent pointer 2 bytes			
optional data 0-40 bytes							

Exemplo:

Transmission Control Protocol (TCP) Header 20-60 bytes



- **Sintaxe** dos tipos de mensagem.

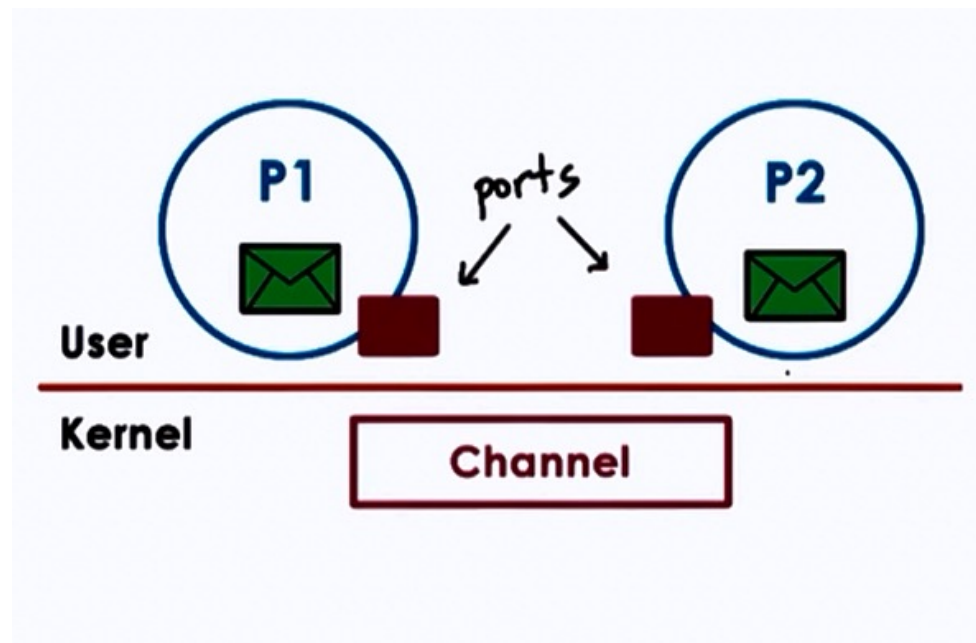
- Define campos da mensagem, e como são **delimitados**.
- Ex: Qtos bits tem os campos porta origem e porta destino, e onde eles começam e onde acabam.

- **Semântica dos campos.**

- Define o **significado da informação** dos campos.
- Ex: O que significa “porta origem” e “porta destino”.

Aplicações de rede: **DEFINIÇÕES: Processo**

- ❑ **Processo** é um programa que roda num *host*.
- ❑ Dois **processos em hosts distintos** se comunicam usando um **protocolo da camada de transporte** → vista mais adiante.



Aplicações de rede: **DEFINIÇÕES: UA – User Agent**

❑ **User Agent - Agente de usuário (UA):**

- É uma **interface** entre o usuário e a aplicação de rede.

❑ Exemplos:

- Navegador web.
- *App* de Correio eletrônico.
- *App* de *Streaming*.
- *Etc...*

Não é o protocolo de aplicação, apesar de conter a implementação do protocolo.

Aplicações de rede: DEFINIÇÕES – Paradigma cliente - servidor

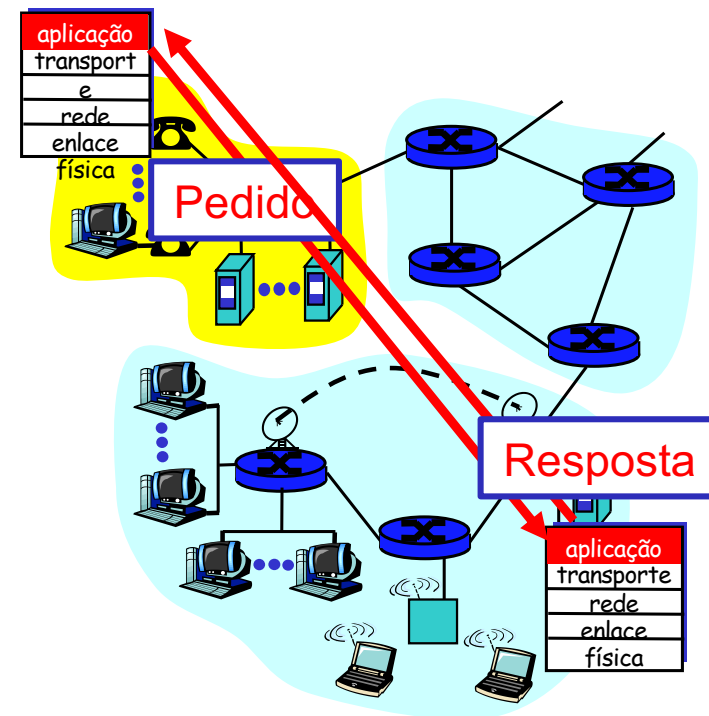
Aplicação de rede típica tem duas partes: **cliente** e **servidor**

Cliente:

- É aquele que “chama”.
- **Solicita** serviço do servidor.

Servidor:

- Fornece o serviço.
- “Ouve” pedidos de conexão.



Dois processos **em hosts distintos** se comunicam usando um **protocolo da camada de transporte**.

API - *Application Program Interface*:

- Interface de programação de aplicações.
 - Define a **interface** entre a **aplicação** e **camada de transporte**.
- ❑ APIs → definidas pelos RFCs.
 - ❑ APIs utilizam os *Sockets*
 - Um socket no sistema operacional é um **canal de comunicação bidirecional que permite a troca de dados entre processos**, tanto localmente quanto através de redes.

Sockets (1)

- ❑ **Socket** funciona como **interface** de entrada e saída do processo:

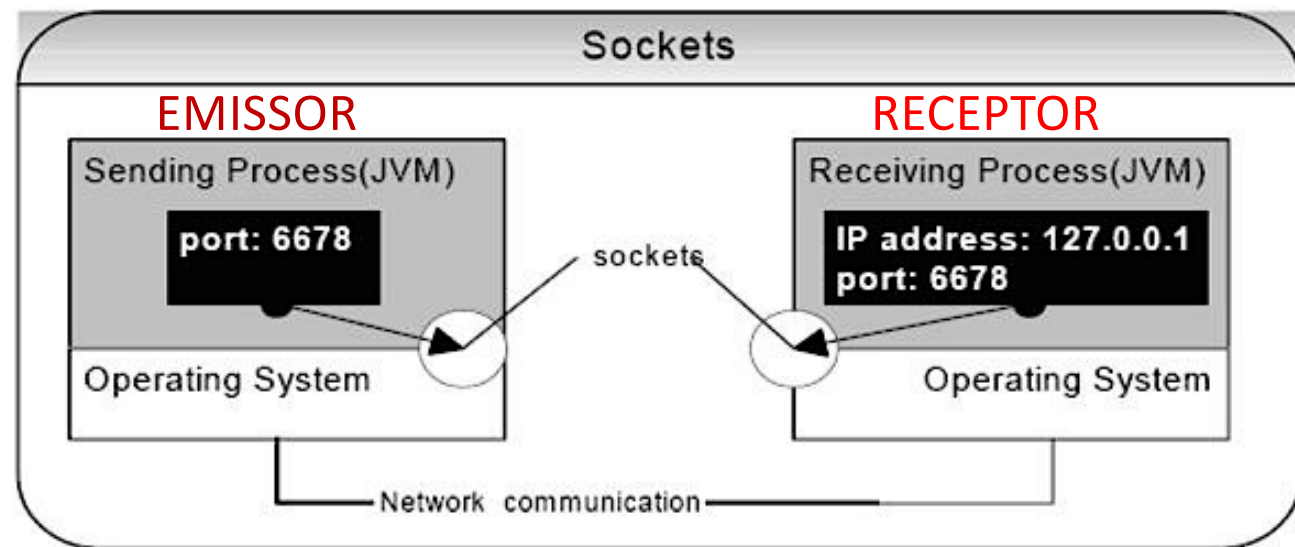
Um processo **emite e recebe mensagens** da rede, **através de seus sockets**.

O processo usa uma **infra-estrutura de transporte**, a qual levará a mensagem até a porta do processo do destino.

O processo não precisa se preocupar **como** a mensagem será transportada.

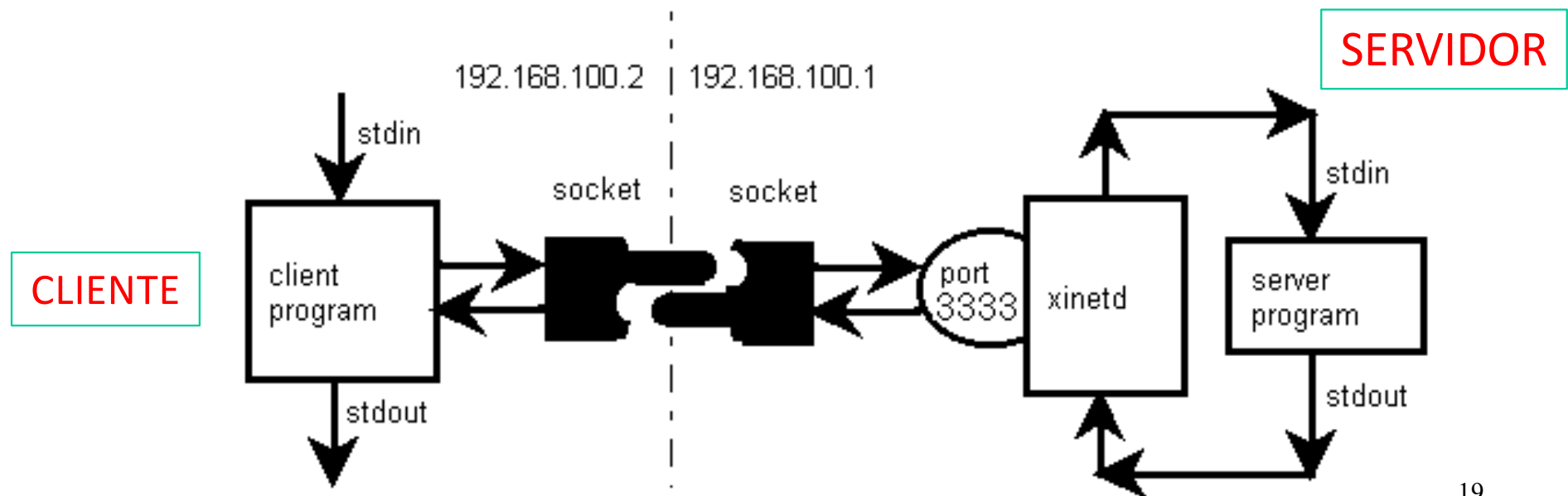
Sockets (2)

- ❑ Quando um **processo** deseja **receber dados de entrada** de uma conexão de rede, **ele cria um socket** e associa esse **socket** a uma **porta**.
- ❑ Quando um **cliente se conecta a esse endereço IP e porta**, o sistema operacional redireciona a conexão para o **processo que está escutando nessa porta**, permitindo que os dados sejam trocados entre o cliente e o processo.



Sockets (3)

- Quando um **cliente se conecta a esse endereço IP e porta**, o sistema operacional redireciona a conexão para o **processo que está escutando nessa porta**.
 - E os dados sejam **trocados entre o cliente e o processo**.



Como um processo identifica outro? (1)

- ❑ **Endereço IP** do *host* que roda o outro processo
 - **IP** : valor de 32-bits que identifica unicamente uma interface.
 - (Pode ser um IPv6 com 64 bits – discutido mais adiante na camada de redes)

10.145.9.9

- ❑ **Número de porta** destino: permite que o receptor determine para qual processo deve ser entregue a mensagem.
 - Exemplo: Porta **80**.

10.145.9.9:80

(RFC 1700 - Portas *well-known* ou *WKS*)

Como um processo identifica outro? (2)

Endereço IP origem (IP_o), **Número de porta** origem (P_o)

+

Endereço IP destino (IP_d), **Número de porta** destino (P_d)

$(IP_o, P_o ; IP_d, P_d)$

Identificam **univocamente** uma comunicação entre dois processos na Internet, numa dada janela de tempo.

Criação e uso de sockets

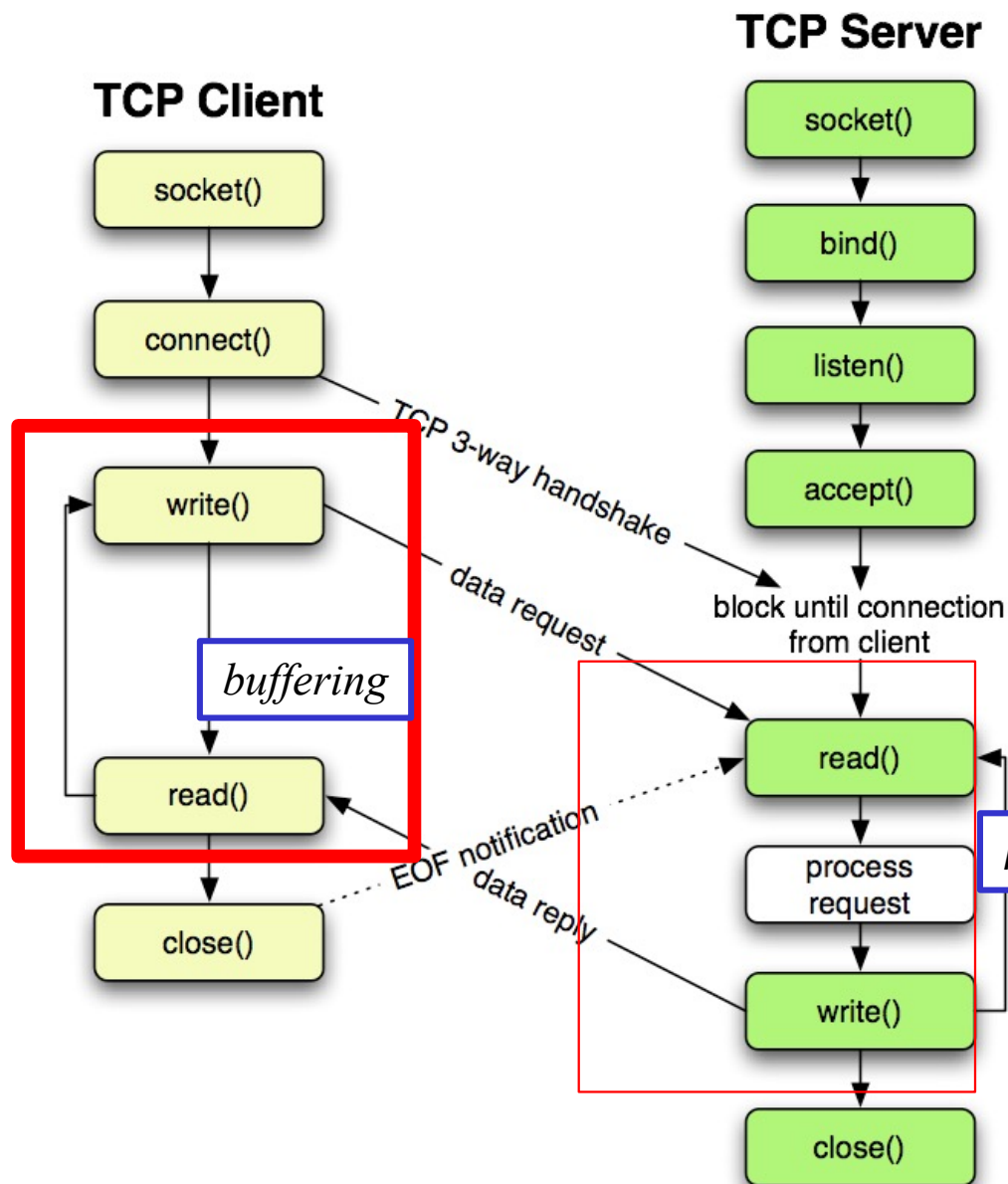
- ❑ **APIs de sockets:** O sistema operacional fornece APIs que permitem aos programadores criar, configurar e utilizar sockets.
 - A API de sockets mais popular é a Berkeley sockets, presente em sistemas Unix, Linux e Windows.
- ❑ **Chamadas de sistema:** As APIs de sockets fornecem chamadas de sistema para operações como criar um socket, conectar-se a outro socket, enviar e receber dados, e fechar um socket.

Não confundir *socket* com porta

- ❑ **Socket** é uma abstração do sistema operacional, utilizada para **representar uma sessão de rede**.
 - Um **socket** é uma interface para programação de aplicativos (API) que **permite que um processo se comunique com outros processos**, por meio de uma rede.
- ❑ **Porta** é um **número associado a um endereço IP** que identifica um processo em execução no sistema que está **aguardando conexões de entrada em uma rede**.
 - Uma **porta** é um número que permite que o sistema operacional saiba **qual processo deve receber os dados de uma conexão de entrada**.

Tipos de Sockets

- ❑ **Sockets de fluxo** (`SOCK_STREAM`): Fornecem um **fluxo de dados confiável e ordenado**, semelhante a um cano.
 - São ideais para comunicação onde a ordem e a integridade dos dados são importantes, como em transferências de arquivos ou conversas online.
- ❑ **Sockets de datagrama** (`SOCK_DGRAM`): Transmitem **mensagens sem garantia de entrega ou ordem**. São mais rápidos que os sockets de fluxo, mas menos confiáveis.
 - São adequados para aplicações onde a velocidade é mais importante que a confiabilidade, como em jogos online ou streaming de mídia.
- A **documentação oficial do kernel Linux** contém **informações detalhadas sobre sockets, incluindo chamadas de sistema, estruturas de dados e protocolos**. Você pode acessá-la online em <https://linux.die.net/man/> ou através do comando `man` no terminal.
- A **biblioteca de sockets do Linux** fornece **APIs para criação, configuração e uso de sockets**. A documentação da API está disponível online em <https://man7.org/linux/man-pages/man2/socket.2.html> e também pode ser acessada através do comando `man` no terminal. - <https://man7.org/linux/man-pages/man2/socket.2.html>



Exemplo de **comunicação lógica** de um processo com conexão TCP, por intermédio dos sockets.

Os principais protocolos de aplicação

HTTP

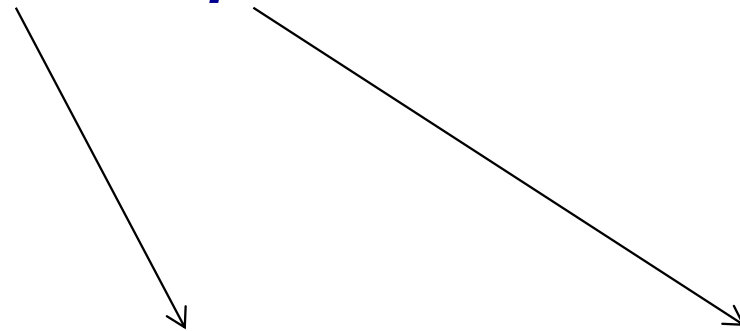
HiperText Transport Protocol

Web: html, clientes e servidores.

□ Páginas WWW:

- Página base HTML,
- **Vários objetos.**

□ URL tem 2 partes: nome do **FQDN**, e **path**.



`www2.unesp.br/images/unesp-full-center.svg`

(FQDN)

□ UA para web:

- *Chrome.*
- *Firefox.*
- *Safari, etc...*

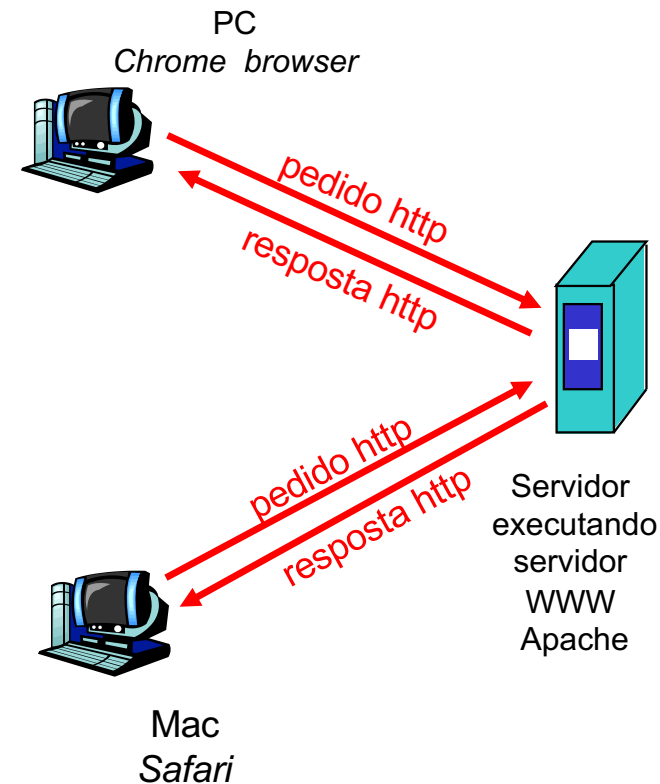
□ Apps Servidores web:

- Apache.
- Tomcat
- NGINX
- *Microsoft Internet Information Server (IIS).*
- Vários outros...

WWW - o protocolo http

http: Hypertext Transfer Protocol

- ❑ Protocolo da **camada de aplicação**.
- ❑ Modelo cliente-servidor
 - *cliente*: solicita e recebe **objetos**.
 - *servidor*: **envia objetos** em resposta a pedidos.
- ❑ ~~http1.0: RFC-1945~~ → **depreciado**
- ❑ http1.1: RFC-2068
 - (visto adiante)



Características do protocolo HTTP:

❑ Usa transporte TCP:

- 1.) Cliente inicia conexão TCP ao servidor, na porta 80.
- 2.) Servidor **aceita conexão**.
- 3.) Troca mensagens http (mensagens do protocolo) entre *browser* (cliente http) e web server (servidor http).
- 4.) **Encerra conexão TCP**.

❑ http é “**sem estado**”. (“**stateless**”)

- ❑ Servidor **não mantém informação** sobre pedidos anteriores do cliente.

Protocolos que mantêm “**estado**” são mais **complexos**.

- São chamados de Protocolos “**Statefull**”.
- O histórico passado (estado) tem que ser mantido.
- Exige mais recursos.

Exemplo de http (1)

Exemplo: Um usuário digita a URL

`http://www.xpto.com.br/index.html`

Exemplo: contém texto e
mais referências a 10
imagens jpeg

1a. **Cliente** http inicia conexão TCP
ao servidor http (processo) em
`www.xpto.com.br`.

Porta 80 é padrão para servidor http.

1b. **Servidor** http no hospedeiro
`www.xpto.com.br` aguarda conexão
TCP na porta 80. “Aceita” conexão,
avisando ao cliente

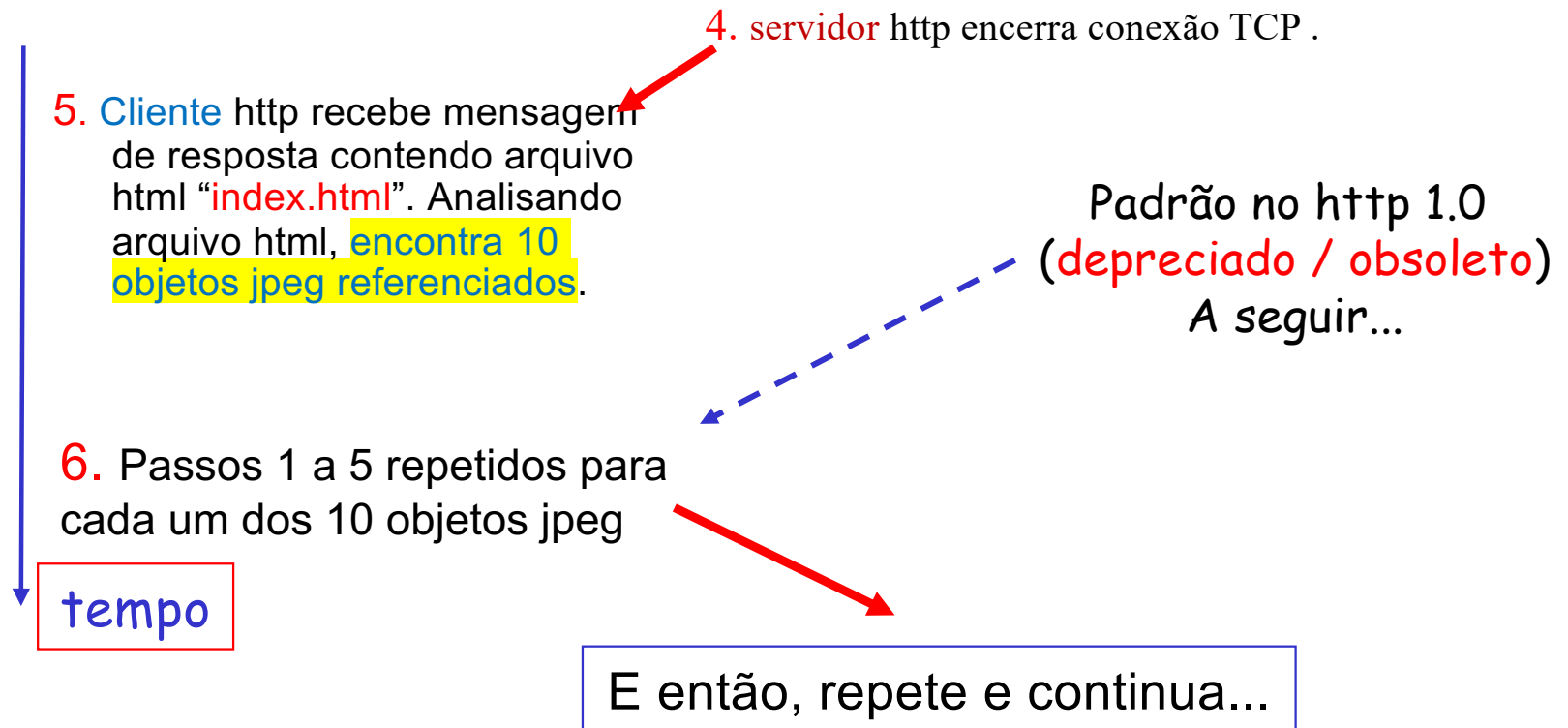
2. **Cliente** http envia *mensagem de
pedido* de http (contendo URL)
através do *socket* da conexão TCP

3. **Servidor** http recebe mensagem de
pedido, elabora *mensagem de resposta*
contendo objeto solicitado
(`index.html`), e envia mensagem via
socket.

tempo

Continua...

Exemplo de http (2)



Conexão http não-persistente (depreciada)

HTTP/1.0 conexão http **não-persistente** (depreciada):

❑ **Cliente:** Abre conexão → solicita objeto

❑ **Servidor:**

Analisa pedido → Responde → **Encerra conexão TCP.**

❑ **Cliente:** Solicita **cada objeto** referenciado na resposta.

- Repete o ciclo para cada objeto.
- 2 RTTs para trazer **cada objeto** (visto em seguida)
(RTT = *round trip time* = tempo de ida e volta).
- Transferência de cada objeto sofre de **partida lenta**.

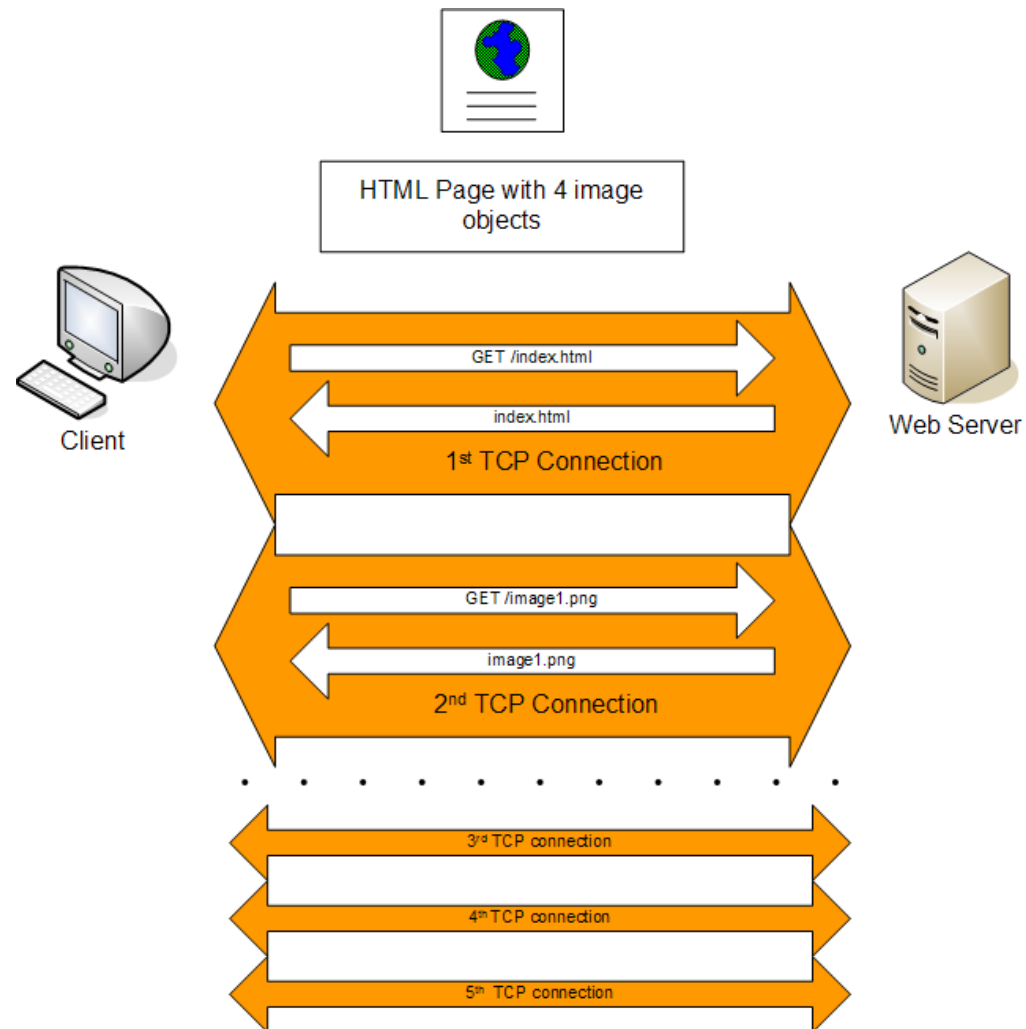
Conceito de *RTT - Round Trip Time* (1)

- ❑ Tempo ***round-trip-time* (RTT)**, é o tempo gasto para um pacote **viajar do cliente ao servidor, e então voltar ao cliente**.
 - **É o tempo de ida e volta cliente - servidor.**
 - **RTT** inclui os atrasos de propagação, de enfileiramento e de processamento em routers e comutadores intermediários.
- ❑ Exemplo numa conexão web: quando se clica num *hyperlink*. Isto faz com que o browser **inicie uma conexão do TCP entre o browser e o web server**.
 - Envolve um “***three-way handshake***” do TCP para estabelecer a conexão.

Conceito de *RTT* - Round Trip Time (2)

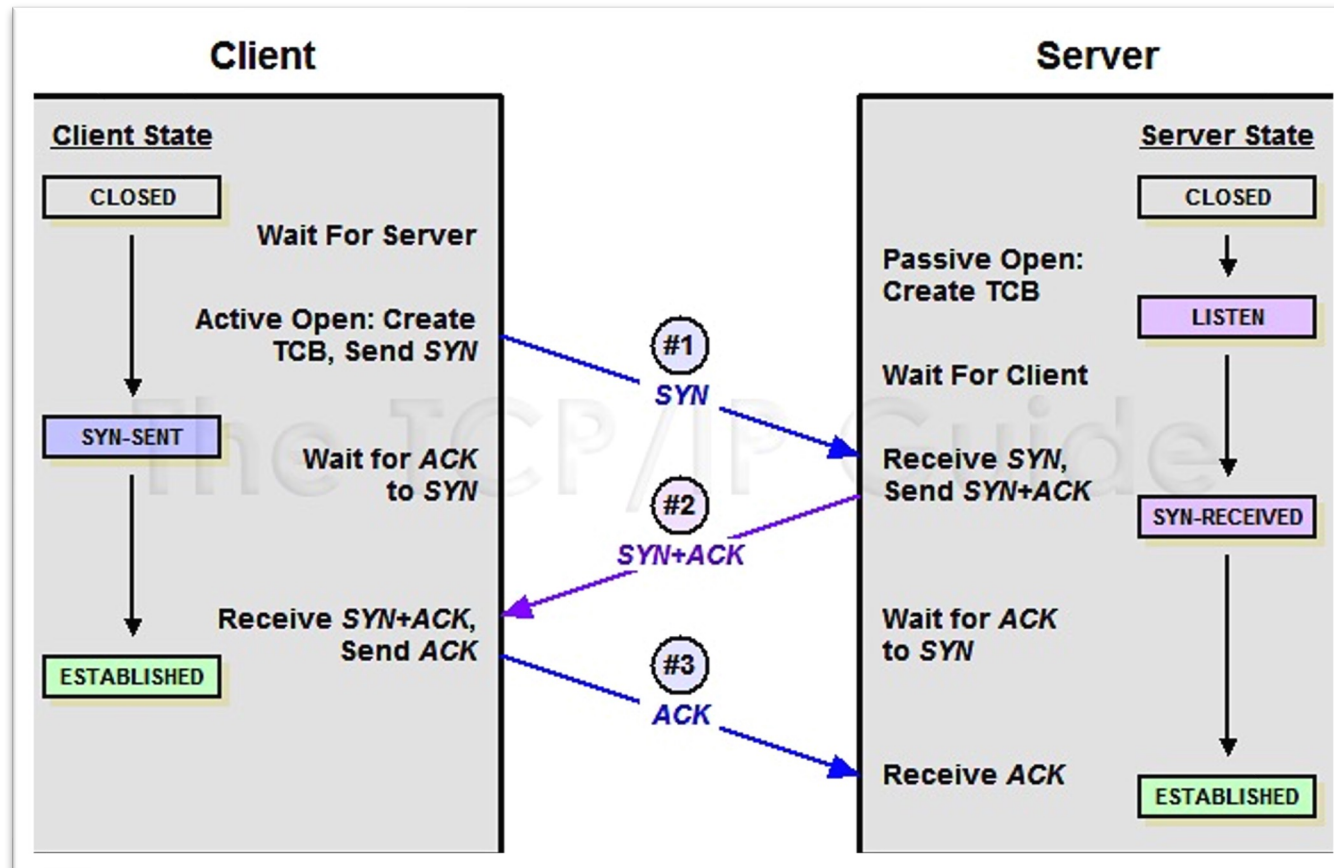
- ❑ Mais um **RTT** decorre após as duas primeiras partes do *three-way handshake*.
 - Após ter terminado as primeiras duas partes do *handshake*, **o cliente emite a mensagem de requisição HTTP na conexão TCP**, e o TCP “agrega” a última confirmação (a terceira parte do *three-way handshake*) na mensagem do pedido. Uma vez a mensagem do pedido chega no usuário, o usuário emite o arquivo HTML na conexão do TCP.
 - Esta interação HTTP “*request-response*” **gasta outro RTT**.
- ❑ Assim, **o tempo de resposta total é 2 RTTs**, **mais o tempo da transmissão do arquivo HTML pelo servidor**.

Conexão http não-persistente (2)



TCP 3-Way Handshaking

Será visto na camada de transporte. Mas, quem quiser pode se antecipar:



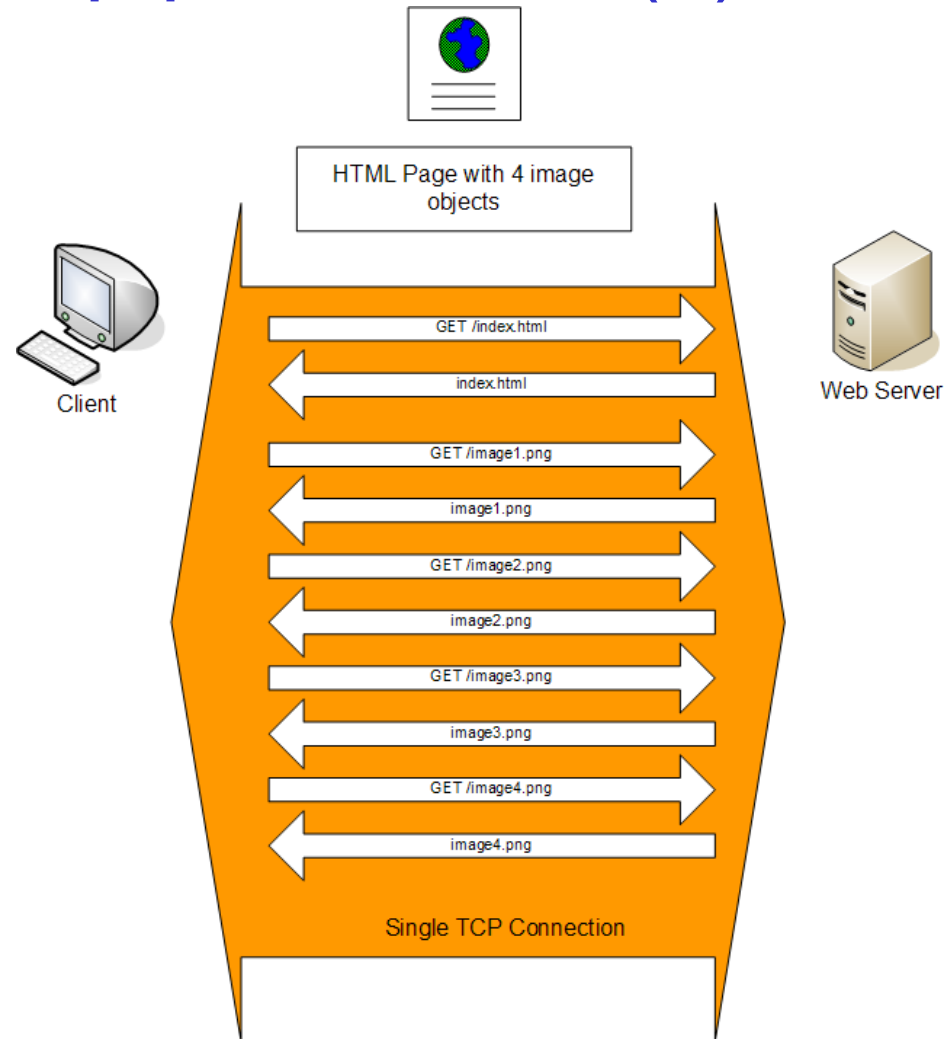
http://www.tcpipguide.com/free/t_TCPConnectionEstablishmentProcessTheThreeWayHandsh-3.htm

HTTP/1.1 - http persistente (1)

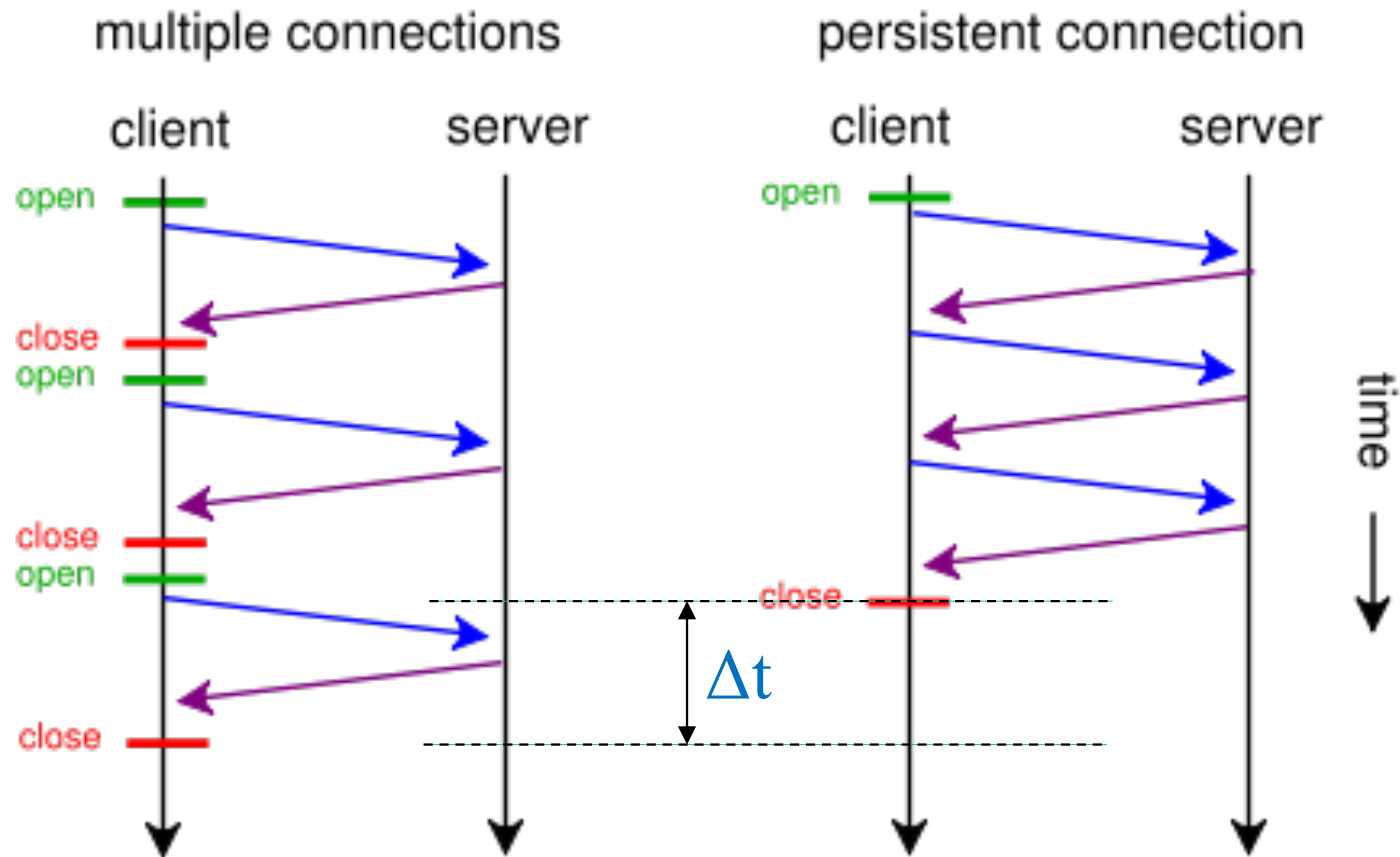
- ❑ Default no HTTP/1.1.
- ❑ **Na mesma conexão** TCP: servidor analisa pedido, responde, analisa novo pedido, etc...
 - **Não fecha a conexão.**
- ❑ **Menos RTTs, e menos partida lenta.**

Os browsers atuais usam conexões TCP paralelas.
(com *pipeline* - veremos em seguida)

HTTP/1.1 - http persistente (2)



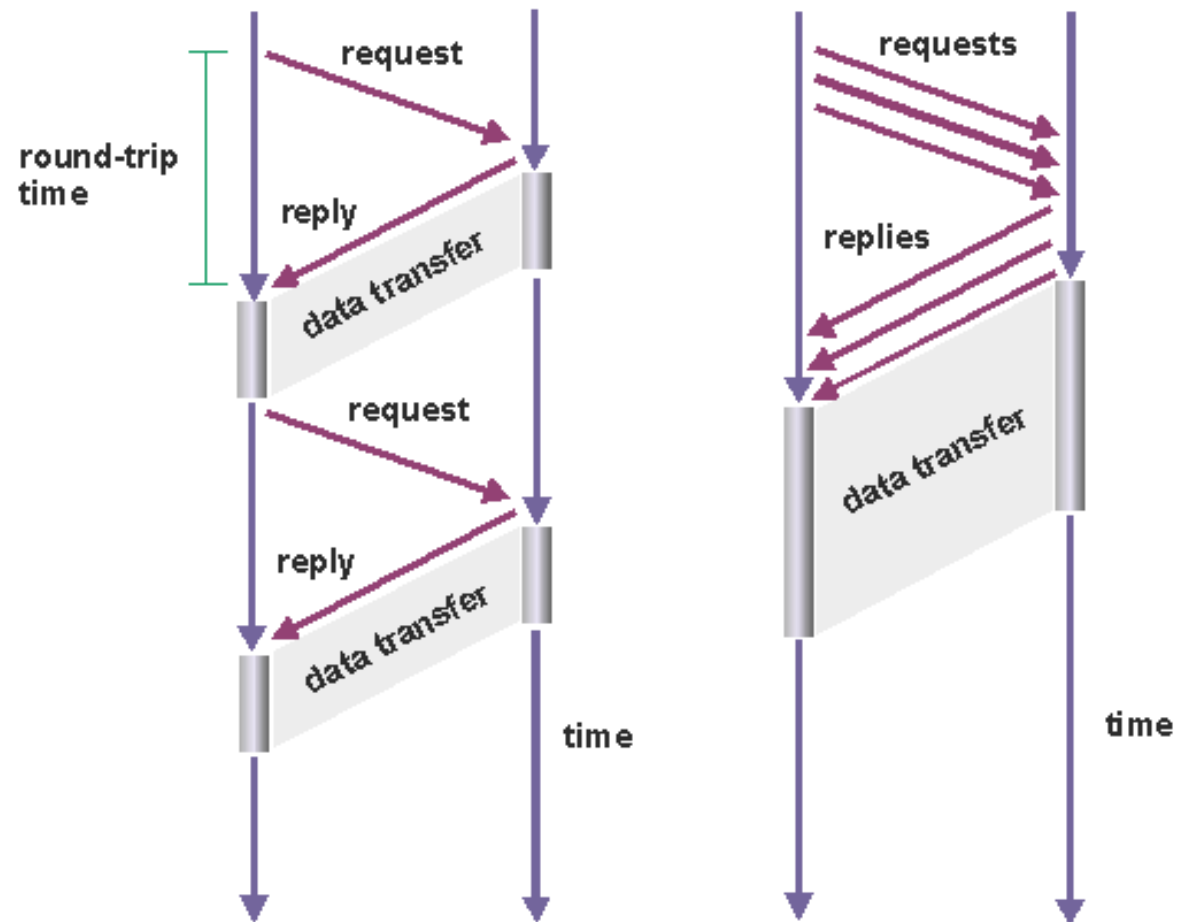
Não-persistente X persistente



Conexão persistente e *Pipelining*

- ❑ Duas versões de conexões persistentes:
 - **sem *pipelining*** (paralelismo) e **com *pipelining***.
- ❑ **Sem *pipelining***:
 - O cliente emite um pedido novo somente quando a resposta precedente foi recebida.
- ❑ **Com *pipelining***:
 - O cliente emite um pedido assim que encontrar uma referência:
 - Ou seja, pede tudo de uma vez.
 - É o default para HTTP/1.1.

http sem pipeline x com pipeline



Formato das Mensagens HTTP

Formato das mensagens HTTP

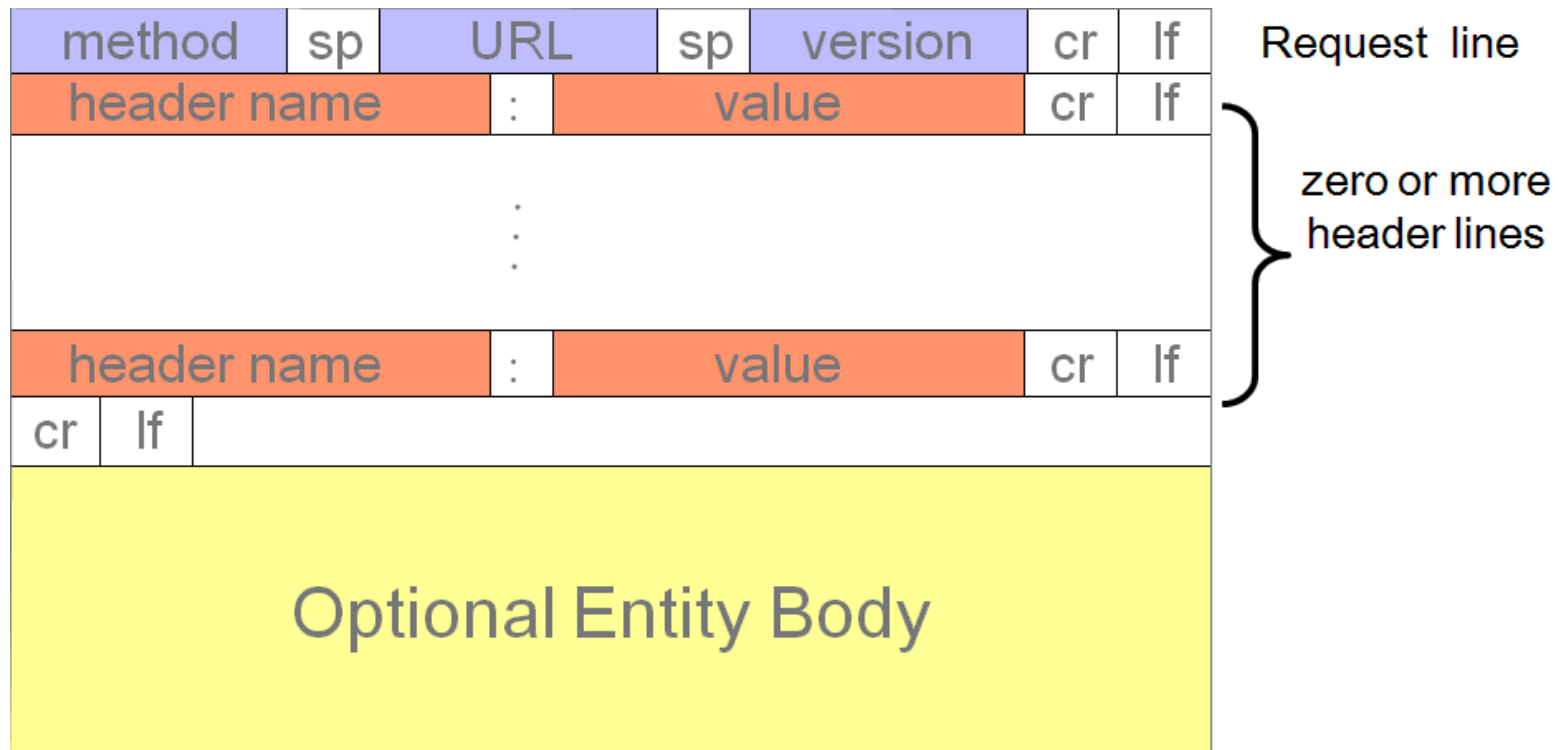
□ Dois tipos de mensagem http:

- **Requisição (*Request*)**
- **Resposta (*Reply*)**

□ **Mensagem http:** formato ASCII

- Conteúdo em ASCII
- Mensagens são legíveis sem necessidade de interpretador.

Mensagem http de **requisição** (1)



Formato geral de uma mensagem de **requisição**

Mensagem http de requisição (2)

linha do pedido
(comandos GET,
POST, HEAD)

GET */dir/page.html* HTTP/1.1

User-agent: Mozilla/4.0

Accept: text/html, image/gif, image/jpeg

Accept-language: en

linhas do
cabeçalho

Carriage return,
line feed
indica fim
de mensagem

(*carriage return* (CR), *line feed* (LF) adicionais)

Mensagem http de requisição (1)

method	sp	URL	sp	version	cr	lf
header name	:	value	cr	lf		
	:					
header name	:	value	cr	lf		
cr	lf					
Optional Entity Body						

GET */dir/page.html* HTTP/1.1

User-agent: Mozilla/4.0

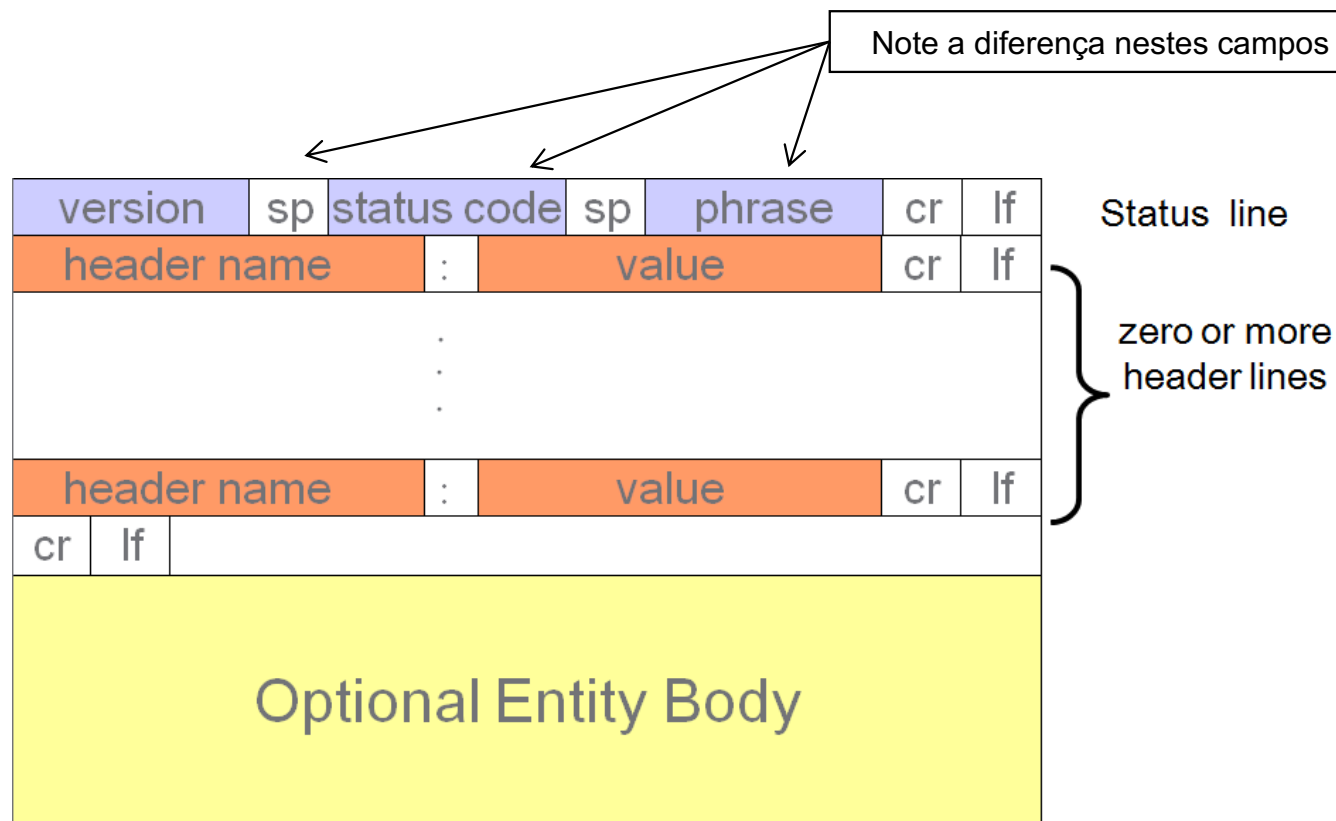
Accept: text/html, image/gif, image/jpeg

Accept-language: en

(carriage return (CR), line feed (LF) adicionais)

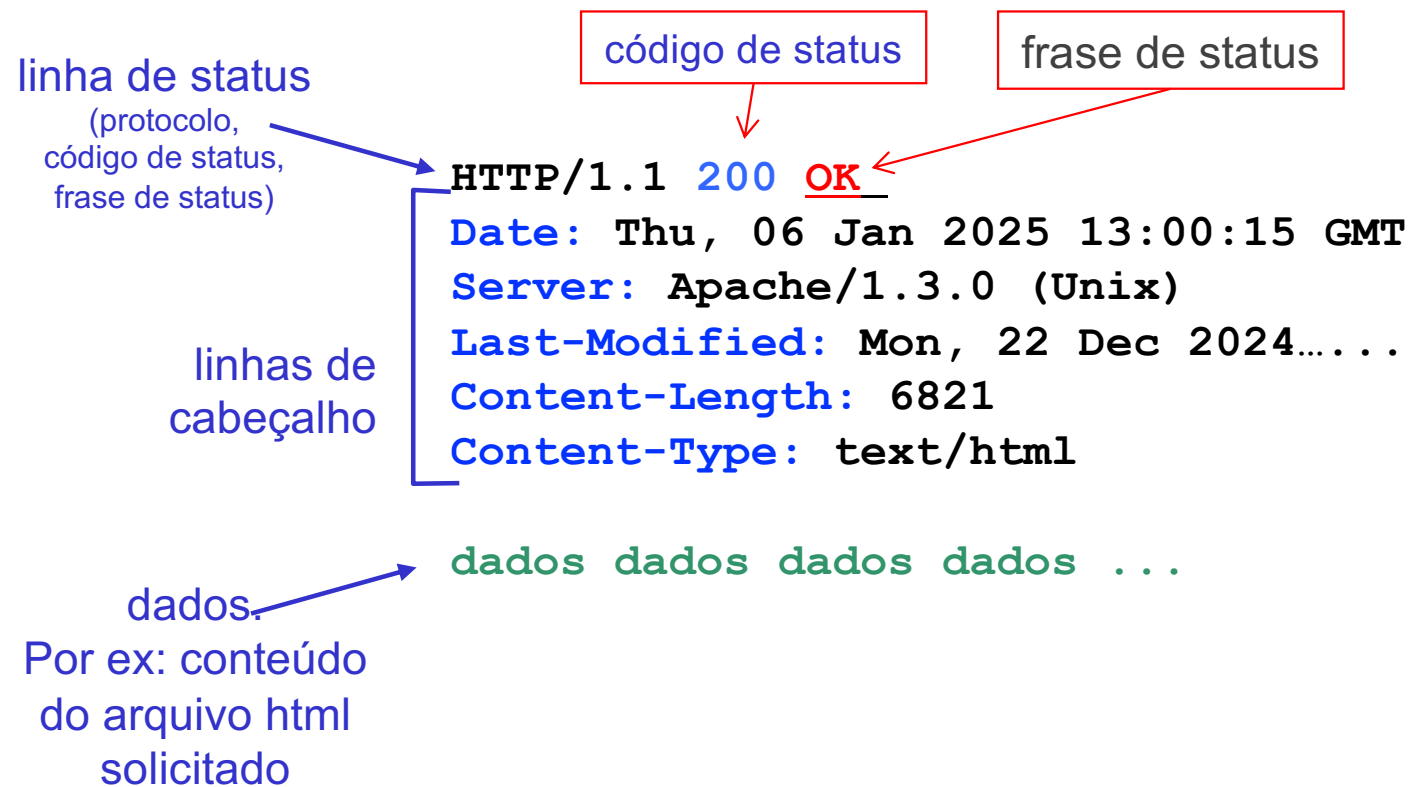
Formato geral de uma mensagem de **requisição**

Mensagem http de resposta (1)



Formato geral de uma mensagem de **RESPOSTA**

Mensagem http de **resposta** (2)



Mensagem http de resposta (3)

version	sp	status code	sp	phrase	cr	lf
header name	:	value	cr	lf		
	:					
header name	:	value	cr	lf		
cr	lf					
Optional Entity Body						

HTTP/1.1 200 OK

Date: Thu, 06 Jan 2025 13:01:10 GMT

Server: Apache/1.3.0 (Unix)

Last-Modified: Mon, 22 Dec 2024....

Content-Length: 6821

Content-Type: text/html

(carriage return (CR), line feed (LF))

dados dados dados dados ...

dados dados dados dados ...

Formato geral de uma mensagem de **RESPOSTA**

Códigos de *status* da resposta http

□ Aparecem na primeira linha da mensagem de resposta.

- Alguns códigos típicos:

200 OK

- Sucesso. Objeto pedido segue mais adiante nesta mensagem.

301 Moved Permanently

- Objeto pedido mudou de lugar, **nova localização especificado nesta mensagem** (*Location:*)

400 Bad Request

- Mensagem de pedido não entendida pelo servidor.

404 Not Found

- Objeto pedido não se encontra neste servidor.

505 HTTP Version Not Supported

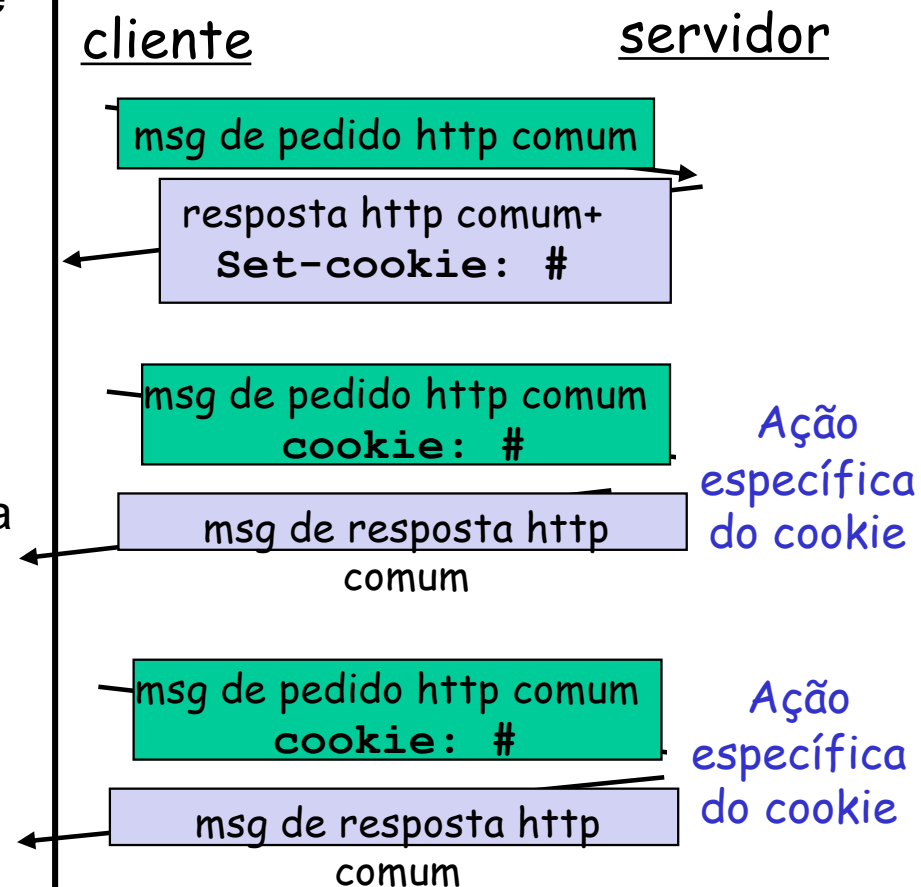
- Versão de http do pedido não aceita por este servidor.

Webservices

- ❑ Solução utilizada para **integração de sistemas** e comunicação entre aplicações diferentes.
 - Permite que novas aplicações possam interagir com outras já existentes.
 - Permite que sistemas desenvolvidos em **plataformas diferentes** sejam compatíveis.
- ❑ São componentes que permitem às aplicações enviar e receber dados.
- ❑ O **objetivo** dos *webservices* é a **comunicação de aplicações através da Internet**.
- ❑ De forma geral os webservices precisam de “**cookies**” para funcionar.

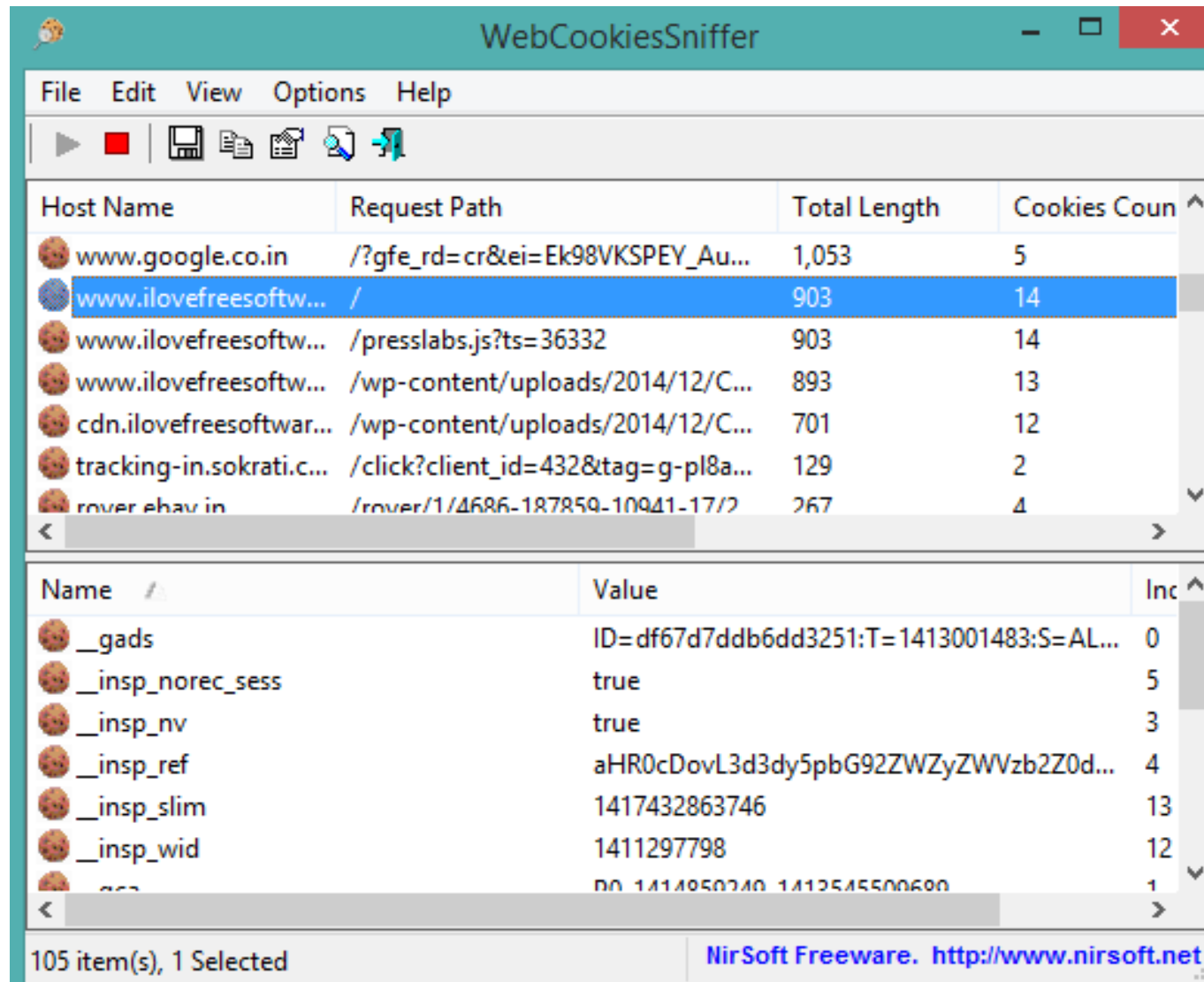
Web: cookies

- ❑ **Servidor** envia “*cookie*” ao cliente na msg de resposta
Set-cookie: 1678453
- ❑ **Cliente** apresenta *cookie* nos pedidos posteriores
cookie: 1678453
- ❑ **Servidor** casa o *cookie* apresentado, com a info guardada no servidor.
 - Autenticação.
 - Lembrando preferências do usuário, opções anteriores, etc...



Consulte quais cookies estão no seu navegador

unesp - SJRP



The screenshot shows the WebCookiesSniffer application window. The top menu bar includes File, Edit, View, Options, and Help. Below the menu is a toolbar with icons for play, stop, save, print, and other functions. The main window displays a table of cookies with columns: Host Name, Request Path, Total Length, and Cookies Count. The second row is selected, showing cookies for www.ilovefreesoftw... with a total length of 903 and 14 cookies. Below this, a detailed view of the selected cookie is shown with columns: Name, Value, and Inc. The first cookie listed is __gads with a value of ID=df67d7ddb6dd3251:T=1413001483:S=AL... and an increment of 0. Other cookies include __insp_norec_sess, __insp_nv, __insp_ref, __insp_slim, __insp_wid, and __insp.

Host Name	Request Path	Total Length	Cookies Count
www.google.co.in	/?gfe_rd=cr&ei=Ek98VKSPEY_Au...	1,053	5
www.ilovefreesoftw...	/	903	14
www.ilovefreesoftw...	/presslabs.js?ts=36332	903	14
www.ilovefreesoftw...	/wp-content/uploads/2014/12/C...	893	13
cdn.ilovefreesoftwar...	/wp-content/uploads/2014/12/C...	701	12
tracking-in.sokrati.c...	/click?client_id=432&tag=g-pl8a...	129	2
rover.ebay.in	/rover/1/4686-187850-10941-17/2	267	4

Name	Value	Inc
__gads	ID=df67d7ddb6dd3251:T=1413001483:S=AL...	0
__insp_norec_sess	true	5
__insp_nv	true	3
__insp_ref	aHR0cDovL3d3dy5pbG92ZWZyZWVzb2Z0d...	4
__insp_slim	1417432863746	13
__insp_wid	1411297798	12
__insp	DO_1414950740_1412545500600	1

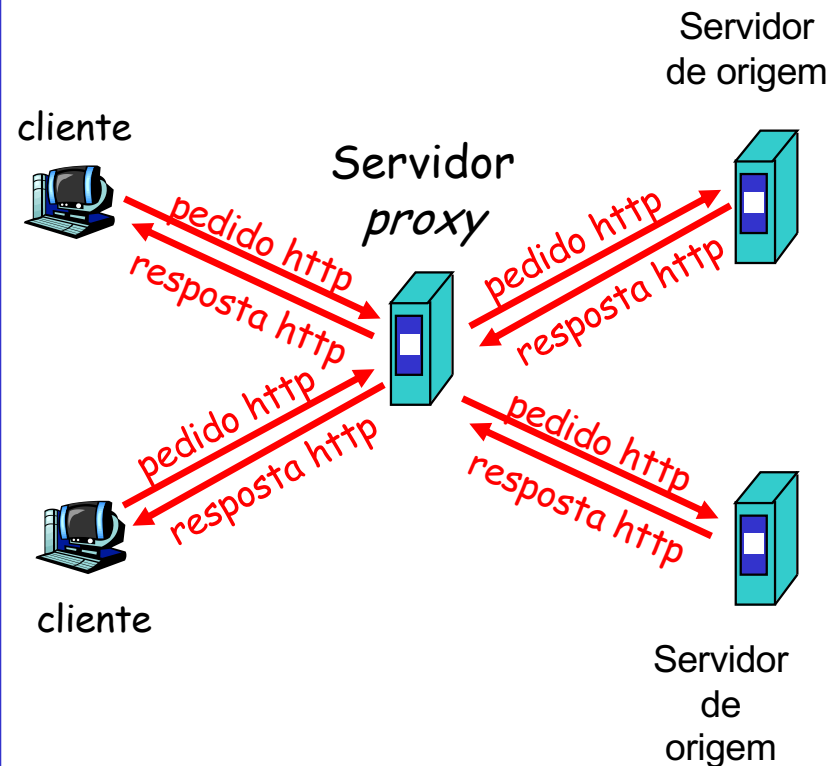
105 item(s), 1 Selected

NirSoft Freeware. <http://www.nirsoft.net>

Proxy

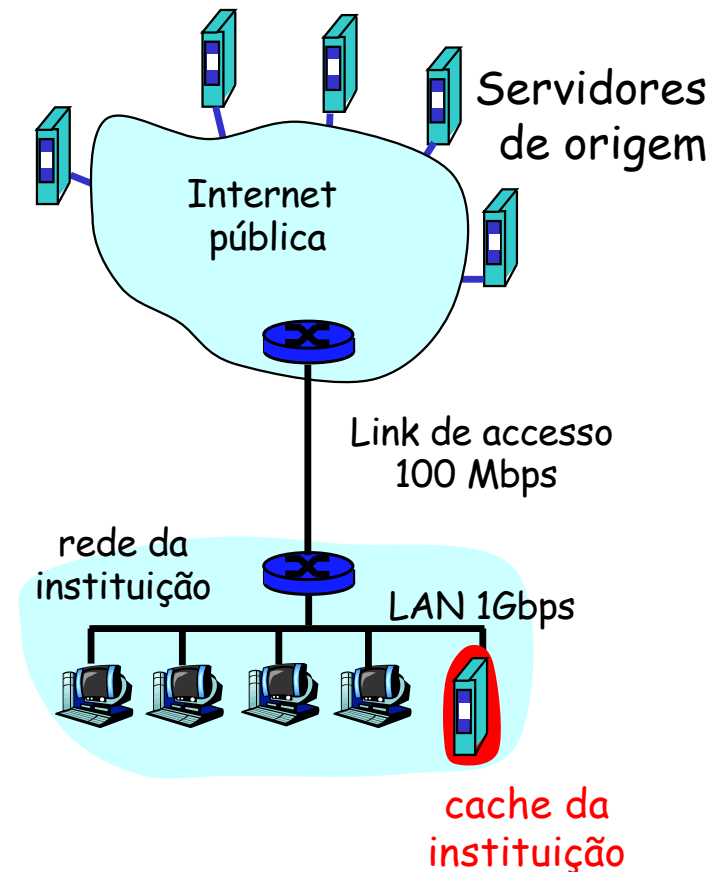
Intermediário entre cliente e o servidor de origem.

- ❑ Cliente envia todos pedidos http ao **proxy**.
 - Se objeto está no cache do procurador, este o devolve imediatamente na resposta http.
 - Senão, solicita objeto do servidor de origem, armazena e depois devolve resposta http ao cliente.



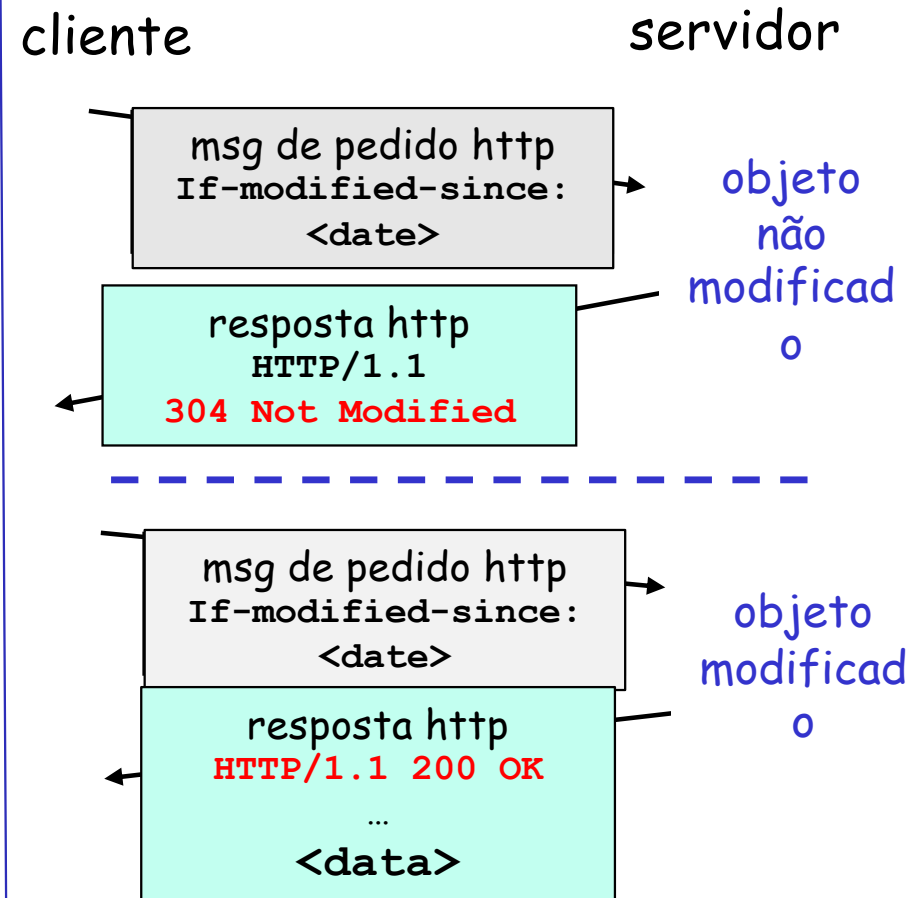
Por que usar cache WWW ?

- ❑ **Suposição:** cache está “próximo” do cliente (por exemplo na mesma rede).
- ❑ **Tempo de resposta menor.**
- ❑ Diminui tráfego aos servidores distantes,
 - Muitas vezes é o gargalo é o enlace (*link*) que liga a rede da instituição até a Internet.



GET condicional e aplicação em proxies

- ❑ **Uso:** não enviar objeto se cliente já tem (*no cache*) versão atual.
- ❑ Cliente: especifica no pedido data da cópia em *cache* http
`If-modified-since: <date>`
- ❑ Servidor: resposta não contém objeto se a cópia no cache é atual:
`HTTP/1.1 304 Not Modified`



HTTP/2 e SPDY

- ❑ Google: fez a proposta do **SPDY** (novembro de 2009).
- ❑ “Camada adicional” no protocolo HTTP.
 - Características originais do protocolo HTTP (cabeçalhos, métodos etc.) são compatíveis.
- ❑ O **SPDY** propôs que cabeçalhos e todos os dados sejam **transmitidos obrigatoriamente compactados e criptografados**.
- ❑ Melhoria de cerca de 40% na performance.
 - Maiores ganhos são registrados em conexões piores.

SPDY e HTTP/2

- ❑ No HTTP 1.1, para melhorar a performance, **habilita-se o GZIP no servidor** para comprimir os dados das respostas (habilitada explicitamente)
 - No **HTTP 2.0**, o **GZIP é padrão e obrigatório**.
- ❑ **SPDY foi descontinuado em favor do http2 (Padronizado em 2015).**
- ❑ No HTTP 2.0, os *headers* são binários e comprimidos usando um algoritmo chamado HPACK.
 - Diminui significativamente o volume de dados trafegados nos *headers*.
- ❑ SPDY **obriga o uso de HTTPS** e conexões seguras.
 - HTTP/2 considera não obrigar ssl (em análise).
 - <https://en.wikipedia.org/wiki/HTTP/2>

<https://www.tunetheweb.com/performance-test/> → permite testar quem performa melhor: HTTP, HTTPS e HTTP/2

HTTP/2

Compatibilidade com browsers em março/2023

HTTP/2 protocol - OTHER

Networking protocol for low-latency transport of content over the web. Originally started out from the SPDY protocol, now standardized as HTTP version 2.

Usage: Global 96.38% + 0.56% = 96.94%

Current aligned Usage relative Date relative Filtered All

Chrome	Edge *	Safari	Firefox	Opera	IE	Chrome for Android	Safari on iOS *	Samsung Internet	Opera Mini *	Opera Mobile *	UC Browser for Android	Android Browser *	Firefox for Android	QQ Browser	Baidu Browser	KaiOS Browser
4-40		3.1-8	2-35	10-27												
41-50	12-18	9-10.1	36-52	28-37			3.2-8.4	4								
51-110	79-110	11-16.2	53-110	38-94	6-10		9-16.2	5-19.0		12-12.1		2.1-4.4.4				2.5
111	111	16.3	111	95	11	111	16.3	20	all	73	13.4	111	110	13.1	13.18	3.1
112-114		16.4-TP	112-113				16.4									

SPDY foi descontinuado em favor do http2

Ver : <http://caniuse.com/#feat=http2>

<http://caniuse.com/#feat=spdy>

© 2025 Prof. Dr. Adriano Mauro Cansian

SPDY

unesp - SJRP

Compatibilidade com browsers em março/2023

SPDY protocol - UNOFF

Networking protocol for low-latency transport of content over the web. Superseded by HTTP version 2.

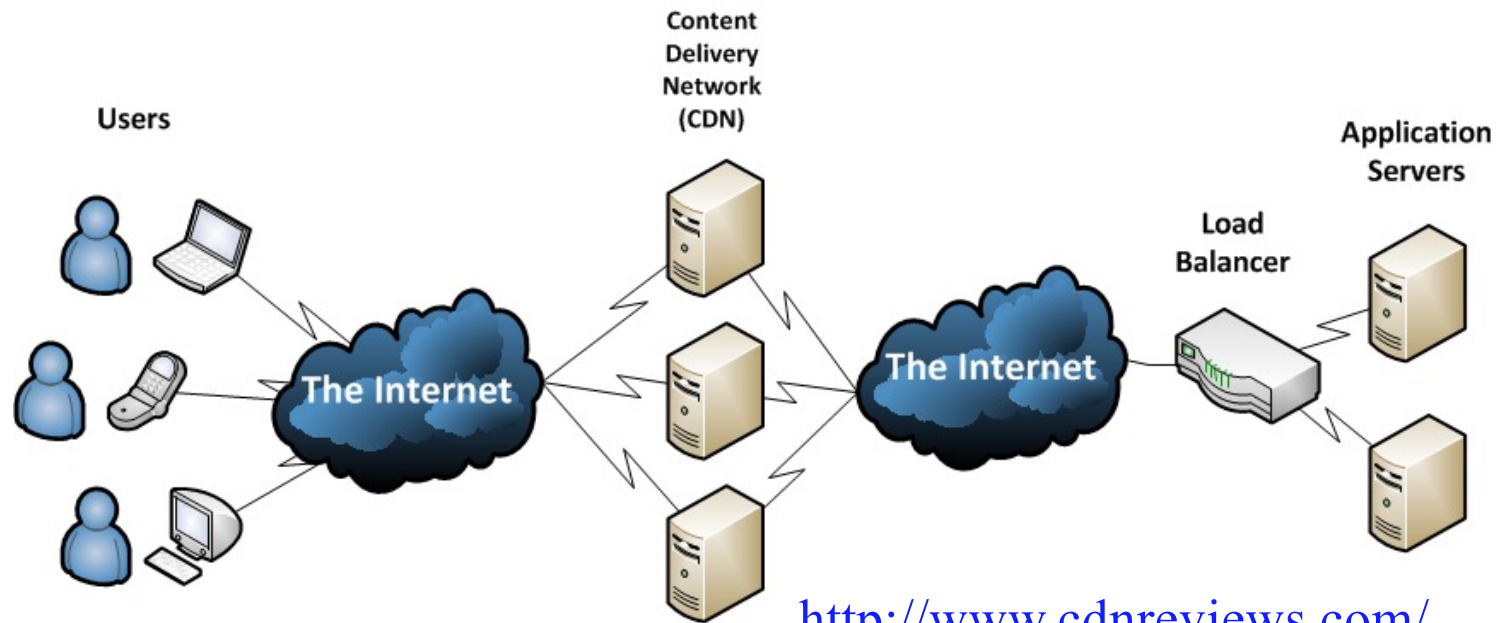
Current aligned Usage relative Date relative Filtered All

Chrome	Edge *	Safari	Firefox	Opera	IE	Chrome for Android	Safari on iOS *	Samsung Internet
				10-11.5				
				12.1-39				
				40-41				
				42				
		3.1-7.1	2-12	43			3.2-7.1	
4-50		8-12	13-50	44			8-12.1	4
51-110	12-110	¹ 12.1-16.2	51-110	45-94	6-10		² 12.2-16.2	5-19.0
111	111	¹ 16.3	111	95	11	111	² 16.3	20
112-114		¹ 16.4-TP	112-113				² 16.4	

Descontinuado

<http://caniuse.com/#feat=spdy>

CDN – Content Delivery Network



maxCDN
CONTENT DELIVERY NETWORK

amazon
cloudfront

CDN77.COM
Content Delivery Network

CLOUDFLARE

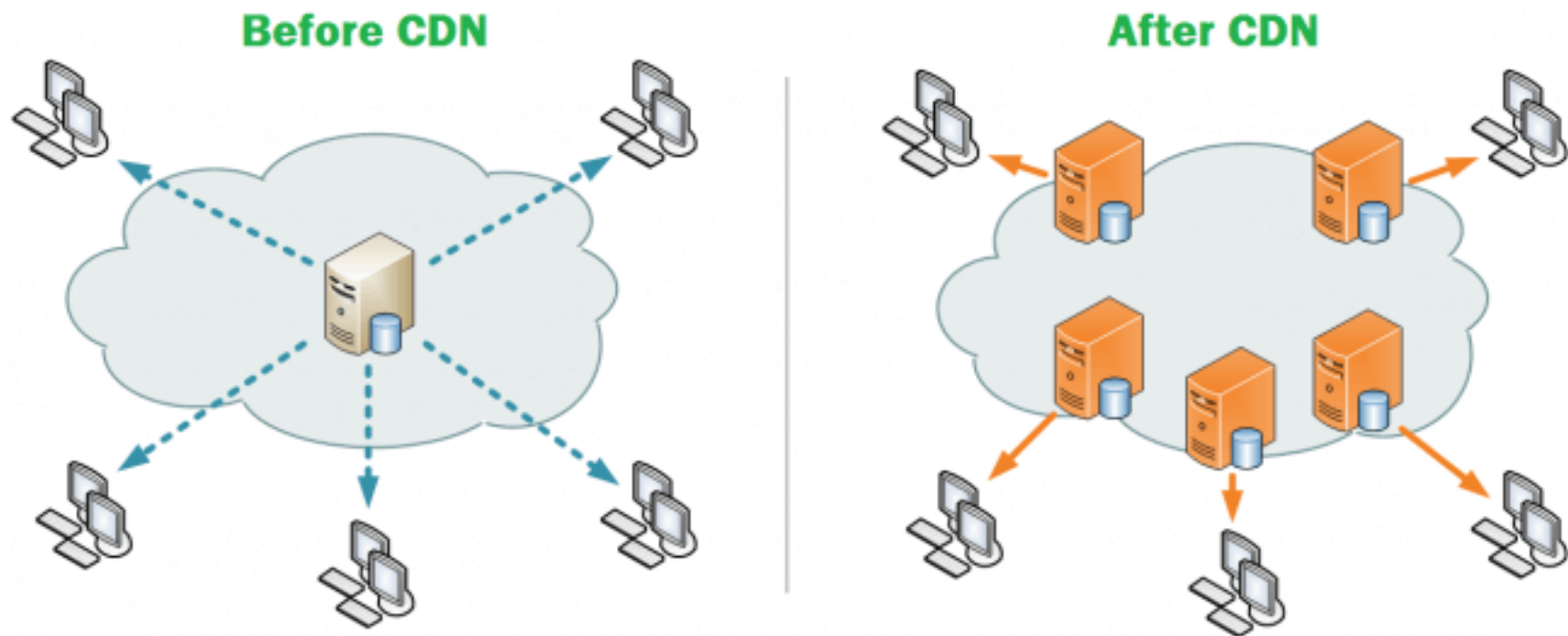
CacheFly

Akamai

CDN – Content Delivery Network

- As CDNs (*Content Delivery Networks*) surgiram para resolver um problema comum na distribuição de conteúdo na internet: **a latência e a velocidade de carregamento**.
- Antigamente, quando um usuário solicitava um conteúdo na internet, como uma página web, por exemplo, a solicitação era encaminhada para o servidor original que hospedava esse conteúdo.
 - Se o servidor estivesse localizado em uma região geográfica distante do usuário, **a resposta demoraria mais para chegar e o tempo de carregamento da página seria maior**.
 - Além disso, se muitos usuários estivessem solicitando o mesmo conteúdo ao mesmo tempo, **o servidor poderia ficar sobrecarregado e demorar ainda mais para responder**.
- As CDNs surgiram para resolver esse problema, distribuindo o conteúdo em servidores espalhados em várias regiões geográficas, de forma que os usuários possam obter o conteúdo de um servidor mais próximo, reduzindo a latência e o tempo de carregamento.
 - A distribuição do conteúdo em vários servidores permite que a carga seja distribuída entre esses servidores, garantindo uma melhor performance e evitando a sobrecarga.

CDN – Content Delivery Network



<http://www.cdnreviews.com/>

Para se aprofundar em http:

- ❑ *Web technology for developers > HTTP*
 - <https://developer.mozilla.org/en-US/docs/Web/HTTP>
- ❑ *HTTP/2 e HTTP/2 Frequently Asked Questions*
 - <https://http2.github.io/>
 - <https://http2.github.io/faq/>

(links verificados em 24 March 2025)

Fim da parte 1...